Белорусский государственный технологический университет
Факультет информационных технологий
Кафедра программной инженерии

# ОТЧЕТ

по лабораторной работе № 3

по дисциплине «Операционные системы»

Управление процессами в Windows и Linux

Выполнил:
Студент 3 курса 9 группы
Антипов Алексей Романович
Преподаватель: Уласевич Н. И.

# СОДЕРЖАНИЕ

Введение ... 3
1 Теоретические основы управления процессами ... 4
1.1 Понятие процесса и его ресурсы ... 4
1.2 Системные интерфейсы Windows и Linux ... 4
2 Создание и исследование процессов Windows ... 6
2.1 Приложение Lab-03x ... 6
2.2 Приложение Lab-03a ... 7
2.3 Глобальная и локальная переменные ITER_NUM ... 9
2.4 Приложение Lab-03b ... 10
2.5 Приложение Lab-03c ... 11
2.6 Наблюдение за процессами через cmd и Диспетчер задач ... 12
3 Анонимные каналы Windows ... 13
3.1 Приложение Lab-03d-client ... 13
3.2 Приложение Lab-03d-server ... 13
4 Передача данных через STARTUPINFO в Windows ... 16
5 Создание и исследование процессов Linux ... 17
5.1 Приложения Lab-03x и Lab-03a ... 17
6 Анонимные каналы Linux ... 19
7 Обработка сигналов Linux ... 21
Заключение ... 23
Список использованных источников ... 24
Приложение А Исходные тексты программ Windows ... 25
Приложение Б Исходные тексты программ Linux ... 38

**Введение**

В этой работе исследованы создание процессов, передача им параметров, наследование ресурсов и обмен данными между родственными процессами. Опыты выполнены в Windows и Ubuntu: результаты сопоставлены с деревом процессов, сведениями операционной системы и исходным кодом программ.

Сначала проверена передача числа итераций через аргументы и переменную окружения. Затем созданы одновременно работающие дочерние процессы, выполнен поиск взаимно простых чисел через анонимный канал и рассмотрены передача строки через STARTUPINFO и обработка сигналов Linux.

– создать дочерние процессы и дождаться их завершения без последовательного выполнения;

– проверить наследование аргументов, переменных окружения и дескрипторов;

– реализовать параллельный поиск взаимно простых чисел через анонимный канал;

– исследовать передачу данных через STARTUPINFO и обработку сигналов POSIX;

– зафиксировать результаты командами, листингами и собственными снимками экрана.

# 1 Теоретические основы управления процессами

## 1.1 Понятие процесса и его ресурсы

Программа представляет собой статический набор инструкций и данных, хранящийся на носителе. Процесс — это динамический экземпляр выполняемой программы. При запуске исполняемого файла операционная система выделяет новому процессу уникальный идентификатор (PID), изолированное виртуальное адресное пространство, процессорное время и другие системные ресурсы.

Архитектура операционных систем поддерживает иерархию процессов, позволяя родительскому процессу создавать дочерние и приостанавливать свое выполнение до их завершения. В ОС Windows для инициализации дочернего процесса применяется функция CreateProcessA. В ОС Linux процесс создания разделен: системный вызов fork дублирует текущий процесс, после чего дочерний процесс замещает свой контекст новой программой с помощью вызова execl.

## 1.2 Системные интерфейсы Windows и Linux

В рамках выполнения лабораторной работы используются системные функции и вызовы, перечень которых приведен в таблице 1.1.

Таблица 1.1 – Средства работы с процессами

| Элемент | Назначение и ожидаемый результат |
|---|---|
| CreateProcessA | Создаёт процесс Windows и возвращает PID и дескрипторы. |
| WaitForMultipleObjects | Ожидает завершения нескольких детей. |
| CreateToolhelp32Snapshot | Создаёт снимок процессов для обхода записей. |
| fork | Создаёт дочерний процесс с новым PID. |
| execl | Загружает другую программу в дочерний процесс. |
| waitpid | Ожидает выбранный дочерний процесс. |
| pipe | Создаёт канал с концами чтения и записи. |
| signal | Назначает обработчик или игнорирование сигнала. |

В обеих операционных системах реализована базовая синхронизация завершения работы: родительский процесс приостанавливает свое

выполнение до успешного завершения всех созданных им дочерних процессов. После возврата управления производится обязательная очистка выделенных системных ресурсов, включая закрытие дескрипторов. Данный подход гарантирует корректное управление полным жизненным циклом процессов и предотвращает утечку ресурсов или появление процессов-сирот в ходе выполнения программы.

# 2 Создание и исследование процессов Windows

## 2.1 Приложение Lab-03x

Приложение Lab-03x выполняет цикл с задержкой 500 мс, выводя на каждом шаге свой PID и номер итерации. Количество шагов задается аргументом командной строки или, при его отсутствии, переменной окружения ITER_NUM. Если оба параметра не заданы, программа выводит сообщение об ошибке и завершается вызовом ExitProcess(1).

Таблица 2.1 – Проверки приложения Lab-03x

| Элемент | Назначение и ожидаемый результат |
|---|---|
| Lab-03x.exe 5 | Пять строк с одним PID и задержкой около 500 мс между строками. |
| set ITER_NUM=4 | Локальная переменная окружения задаёт резервное число итераций. |
| Lab-03x.exe | При наличии ITER_NUM выполняется заданное переменной число итераций. |
| set ITER_NUM= | При отсутствии обоих источников возвращается код ошибки. |

Для подтверждения алгоритма произведено три контрольных запуска: с явной передачей аргумента, с предварительно установленной переменной и без указания параметров. Консольный вывод программы, демонстрирующий корректную обработку приоритетов источников и успешную генерацию ошибки с кодом завершения 1, представлен на рисунке 2.1.

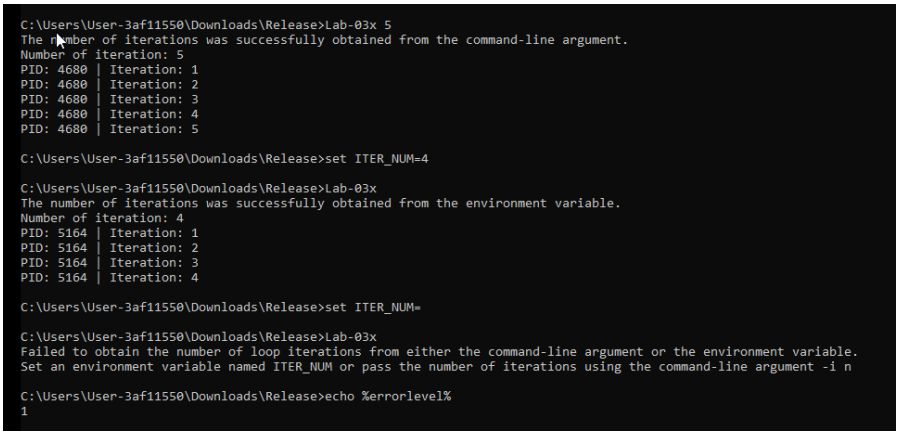



Рисунок 2.1 – Запуски Lab-03x с аргументом, переменной ITER_NUM и ошибкой

Первые два запуска вывели соответственно пять и четыре строки с идентификаторами PID. В последнем случае программа штатно завершила работу с кодом возврата 1.

## 2.2 Приложение Lab-03a

Приложение Lab-03a демонстрирует особенности создания дочерних процессов с помощью системной функции CreateProcessA. В программе реализовано три варианта вызова этой функции. Родительский процесс переходит в режим ожидания только после итеративной попытки запуска всех дочерних экземпляров. Успешно созданные процессы выполняются параллельно.

Спецификация передаваемых параметров CreateProcessA и ожидаемое поведение системы представлены в таблице 2.2. В первом варианте аргумент намеренно объединен с именем модуля, что приводит к системной ошибке. Во втором варианте имя модуля, как и аргумент передаются в качестве 2 аргумента функции. В третьем варианте имя исполняемого файла и его аргументы передаются в функцию раздельно.

Таблица 2.2 – Варианты вызова CreateProcessA

| Элемент | Назначение и ожидаемый результат |
|---|---|
| lpApplicationName = "Lab-03x.exe 50" lpCommandLine = "" | Сбой ERROR_FILE_NOT_FOUND: lpApplicationName должен содержать путь к модулю без аргументов. |
| lpApplicationName = "", lpCommandLine = "Lab-03x.exe 50" | Имя модуля извлекается из первого токена изменяемой командной строки; процесс создаётся. |
| lpApplicationName = "Lab-03x.exe", lpCommandLine = " 50" | Ведущий пробел оставляет пустой argv[0], а число попадает в argv[1]; процесс создаётся. |

Согласно алгоритму работы приложения, первый вызов функции CreateProcessA прогнозируемо завершается системной ошибкой (ERROR_FILE_NOT_FOUND). Это обусловлено намеренно некорректным форматом передачи параметров — объединением имени исполняемого модуля и аргументов в единую строку, а также передачи этой строки в функцию CreateProcessA в качестве первого аргумента. Второй и третий вызовы используют правильное разделение параметров и успешно инициализируют дочерние процессы. Постоянный контроль кодов возврата функции CreateProcessA позволяет надежно дифференцировать программно ожидаемую ошибку от непредвиденных системных сбоев на этапе выполнения.

Консольный вывод родительского приложения, демонстрирующий описанную логику взаимодействия, представлен на рисунке 2.2. Как и предполагалось, первая попытка создания процесса завершилась отказом с кодом системной ошибки 2. В свою очередь, второй и третий процессы были успешно инициализированы операционной системой, о чем достоверно

свидетельствует вывод их уникальных системных идентификаторов (PID) в стандартный поток вывода.

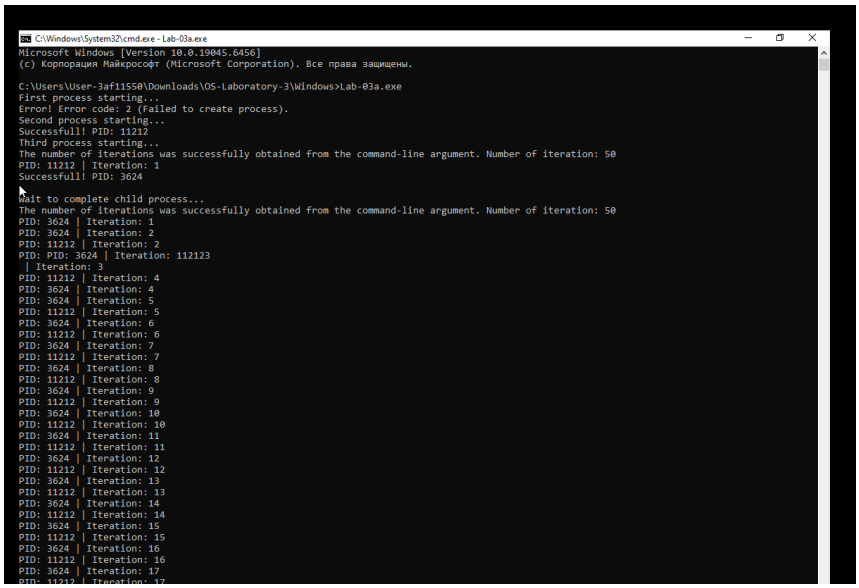



Рисунок 2.2 – Создание дочерних процессов приложением Lab-03a

Оба успешно созданных дочерних процесса отработали заданные 50 итераций. Вследствие параллельного выполнения и использования общего стандартного потока вывода, текстовые строки от разных процессов в консоли чередуются. Подобное наложение вывода является стандартным поведением при отсутствии механизмов межпроцессной синхронизации и не влияет на фактическое количество выполненных итераций.

Раскрытая ветвь Lab-03a в Process Explorer, во время выполнения программы представлена на рисунке 2.3.

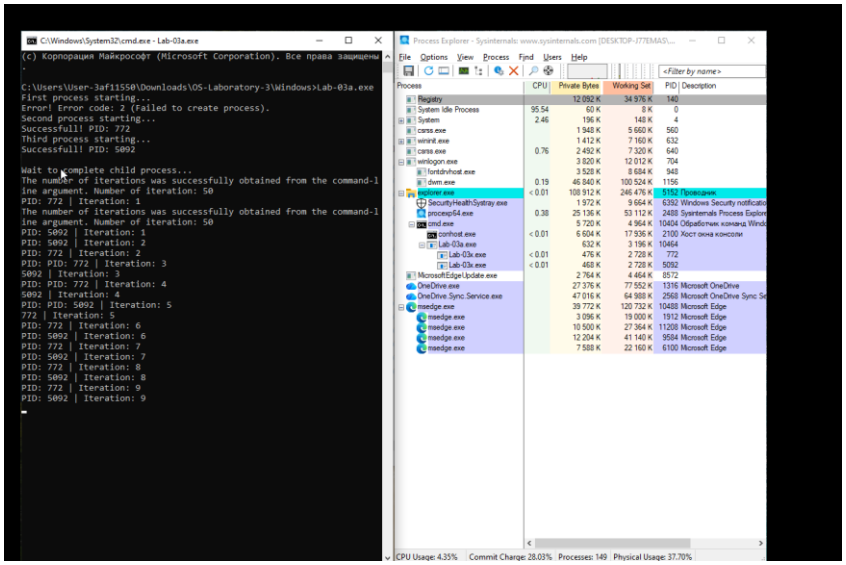



Рисунок 2.3 – Дерево процессов Lab-03a в Process Explorer

В дереве видны два Lab-03x с разными PID под одним Lab-03a. Так подтверждается родительская связь и перекрытие времени работы дочерних процессов.

## 2.3 Глобальная и локальная переменные ITER_NUM

Для тестирования механизма наследования переменных окружения в системе предварительно задано глобальное пользовательское значение ITER_NUM=12. В приложении Lab-03b реализовано сравнение приоритетов источников данных: унаследованная глобальная переменная сопоставляется с переданным аргументом командной строки и локальной переменной со значением 60, инициализированной непосредственно внутри родительского процесса. Перечень задействованных команд и системных функций представлен в таблице 2.3.

Таблица 2.3 – Работа с переменной окружения

| Элемент | Назначение и ожидаемый результат |
|---|---|
| setx ITER_NUM 12 | Сохраняет пользовательское значение для новых процессов. |
| set ITER_NUM | Показывает значение в новой cmd. |
| SetEnvironmentVariableA | Устанавливает локальное ITER_NUM=60 в родительском Lab-03b. |

Мониторинг выделенных системных ресурсов осуществляется посредством утилиты Process Explorer. Инструмент применяется для визуализации иерархии созданных процессов и контроля открытых дескрипторов (Handles), удерживаемых родительским приложением после вызова функции CreateProcessA. Состояние панели дескрипторов для процесса Lab-03a зафиксировано на рисунке 2.4.

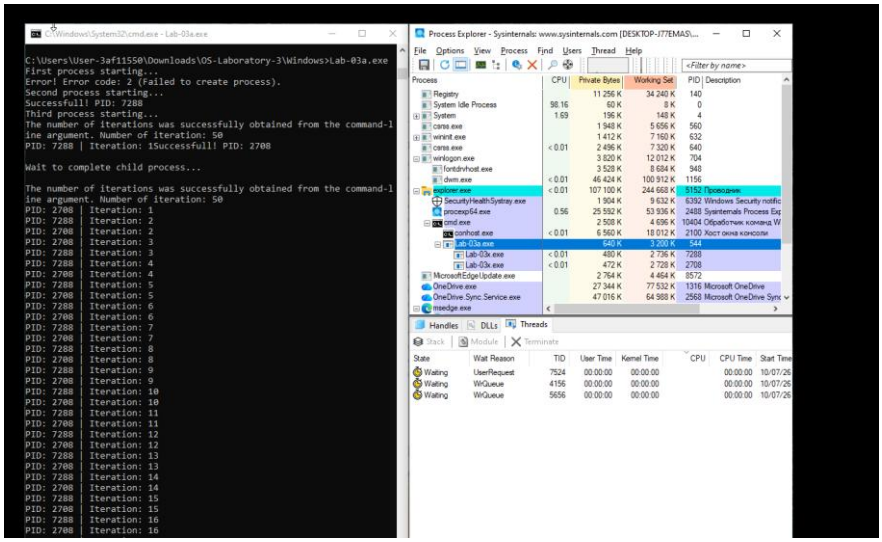



Рисунок 2.4 – Дескрипторы дочерних процессов у Lab-03a

В процессе мониторинга у родительского приложения фиксируются открытые дескрипторы типа Process, ссылающиеся на созданные дочерние процессы. Поскольку после завершения ожидания программа освобождает

выделенные ресурсы вызовом функции CloseHandle, снимок состояния в утилите Process Explorer выполнялся непосредственно во время параллельной работы дочерних процессов.

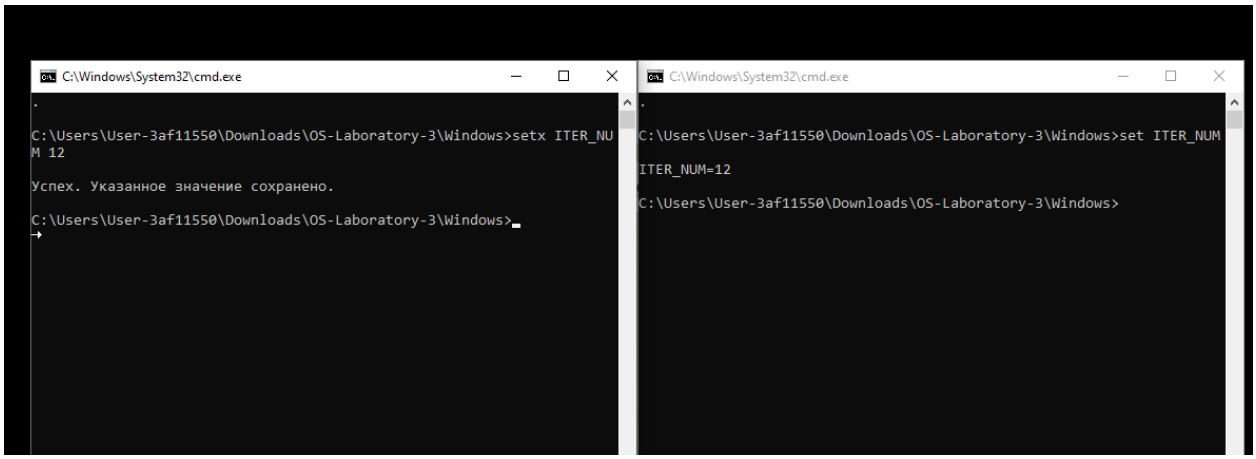



Рисунок 2.5 – Пользовательская переменная окружения ITER_NUM

Для настройки среды выполнения команда setx инициализировала пользовательскую переменную окружения ITER_NUM=12. Проверка применения параметров продемонстрирована на рисунке 2.5. Новый экземпляр командной строки корректно унаследовал заданное значение. В связи с тем, что активные сессии консоли не обновляют переменные окружения динамически, для тестирования приложения Lab-03b использовалось новое окно.

## 2.4 Приложение Lab-03b

Приложение Lab-03b реализует три сценария инициализации дочерних процессов, демонстрируя механизмы передачи параметров и наследования среды выполнения. Первый процесс получает количество итераций из предварительно заданной пользовательской переменной окружения ITER_NUM=12. Второму процессу значение (50 итераций) передается явно через аргумент командной строки. Перед запуском третьего процесса родительское приложение переопределяет среду, устанавливая локальное значение ITER_NUM=60. Вызов функций CreateProcessA для всех экземпляров осуществляется до перехода родительского потока в режим синхронного ожидания их завершения.

Результаты работы дочерних процессов направляются в общий консольный буфер родителя. Фрагмент вывода, подтверждающий успешный старт всех экземпляров и корректное определение источников параметров итерации, представлен на рисунке 2.6.




Рисунок 2.6 – Количество итераций дочерних процессов Lab-03b

Инициализированные дочерние процессы успешно отработали 12, 50 и 60 итераций соответственно. Присвоенные операционной системой уникальные идентификаторы (PID) подтверждают параллельную работу трех независимых процессов. Наблюдаемое чередование выводимых строк в консоли обусловлено асинхронным доступом процессов к общему стандартному потоку вывода при отсутствии механизмов межпроцессной синхронизации.

## 2.5 Приложение Lab-03c

Приложение Lab-03c предназначено для сбора информации о запущенных в системе процессах с использованием интерфейса API Tool Help Library. Программа создает снимок текущего состояния системы с помощью функции CreateToolhelp32Snapshot, применяя системный флаг TH32CS_SNAPPROCESS. Далее выполняется итеративный обход записей посредством вызовов Process32First и Process32Next, которые последовательно считывают данные в структуру PROCESSENTRY32. Для каждого обнаруженного процесса в консоль выводится его исполняемое имя, собственный идентификатор (PID) и идентификатор родительского процесса (PPID). По завершении цикла обхода дескриптор снимка закрывается вызовом CloseHandle для предотвращения утечек памяти и освобождения выделенных ресурсов.

Начало сформированного программой списка процессов приведено на рисунке 2.7.

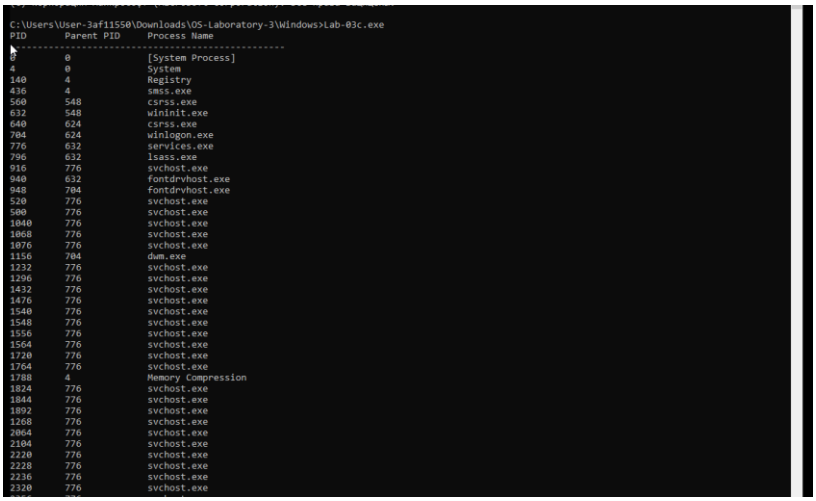



Рисунок 2.7 – Список процессов, сформированный Lab-03c

Каждая запись включает три параметра: PID процесса, Parent PID и имя модуля. Отличия в родительских PID позволяют отделять системные процессы от пользовательских и сверять консольный вывод с деревом процессов в утилите Process Explorer.

## 2.6 Наблюдение за процессами через cmd и Диспетчер задач

Для верификации PID приложения Lab-03a применялась утилита tasklist и вкладка «Подробности» Диспетчера задач. Результаты мониторинга из этих инструментов сопоставлены на рисунке 2.8.

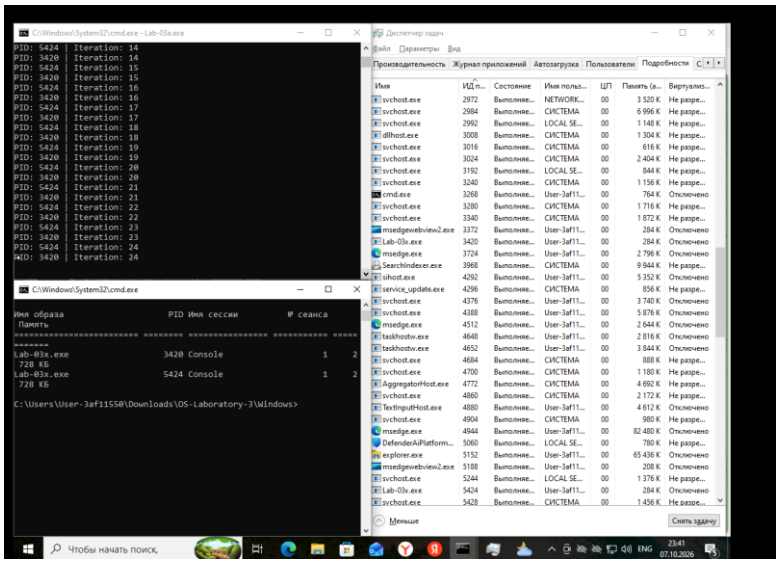



Рисунок 2.8 – Просмотр дочерних процессов в cmd и Диспетчере задач

PID 3420 и 5424 совпадают в выводе tasklist и в Диспетчере задач. Это подтверждает, что наблюдались именно два ребёнка текущего запуска Lab-03a.

# 3 Анонимные каналы Windows

## 3.1 Приложение Lab-03d-client

Клиент Lab-03d-client получает M, L и R из командной строки, перебирает числа диапазона и с помощью алгоритма Евклида оставляет те, для которых НОД(n, M)=1. Найденные значения сначала собираются в строковый буфер, затем одним вызовом WriteFile записываются в стандартный вывод, подключённый родителем к каналу.

## 3.2 Приложение Lab-03d-server

Сервер создаёт один анонимный канал и запускает K клиентов. Записывающий дескриптор передаётся им через поле hStdOutput структуры STARTUPINFOA; читающий дескриптор у детей не наследуется. После запуска всех процессов родитель закрывает свой конец записи, считывает общий поток, ожидает детей и закрывает оставшиеся дескрипторы. Это соответствует реализации в приложении, хотя при большом объёме данных единый канал затрудняет просмотр промежуточных результатов.

Последовательность работы сервера и используемые системные функции приведены в таблице 3.1.

Таблица 3.1 – Этапы параллельного поиска

| Элемент | Назначение и ожидаемый результат |
|---|---|
| CreatePipe | Создаёт один анонимный канал для всех клиентов. |
| SetHandleInformation | Отключает наследование читающего дескриптора. |
| STARTF_USESTDHANDLES | Передаёт записывающий дескриптор как стандартный вывод клиента. |
| WaitForMultipleObjects | Ожидает завершения созданных клиентов. |
| std::chrono | Измеряет время работы параллельного поиска. |

Для проверки результата удобно выбрать небольшой диапазон: он позволяет увидеть параметры запуска, все найденные числа и время работы на одном экране.

При M=12, K=4 и диапазоне [1, 31] итоговый вывод сервера представлен на рисунке 3.1.

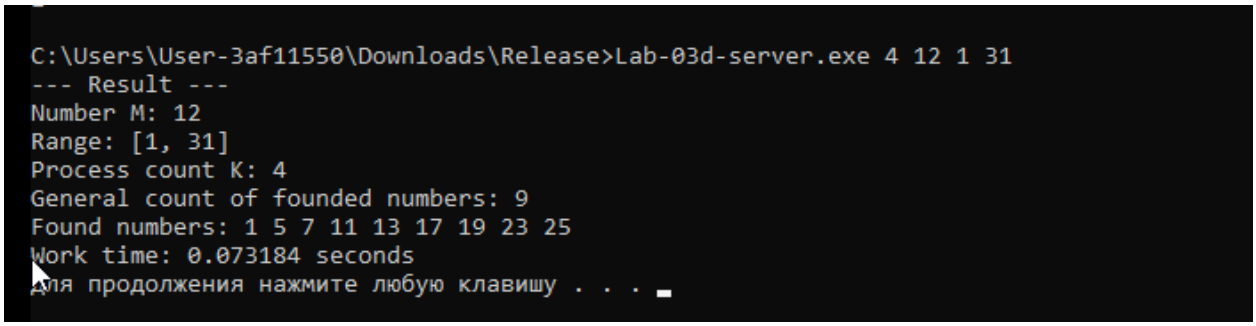



Рисунок 3.1 – Результат Lab-03d-server для нескольких процессов

В заданном диапазоне [1, 31] алгоритм должен определить 11 чисел, взаимно простых с 12: 1, 5, 7, 11, 13, 17, 19, 23, 25, 29 и 31. Итоговое количество найденных значений должно строго соответствовать числу элементов в выведенном массиве.

Для мониторинга иерархии процессов на этапе вычислений применялась системная утилита Process Explorer (рисунок 3.2).

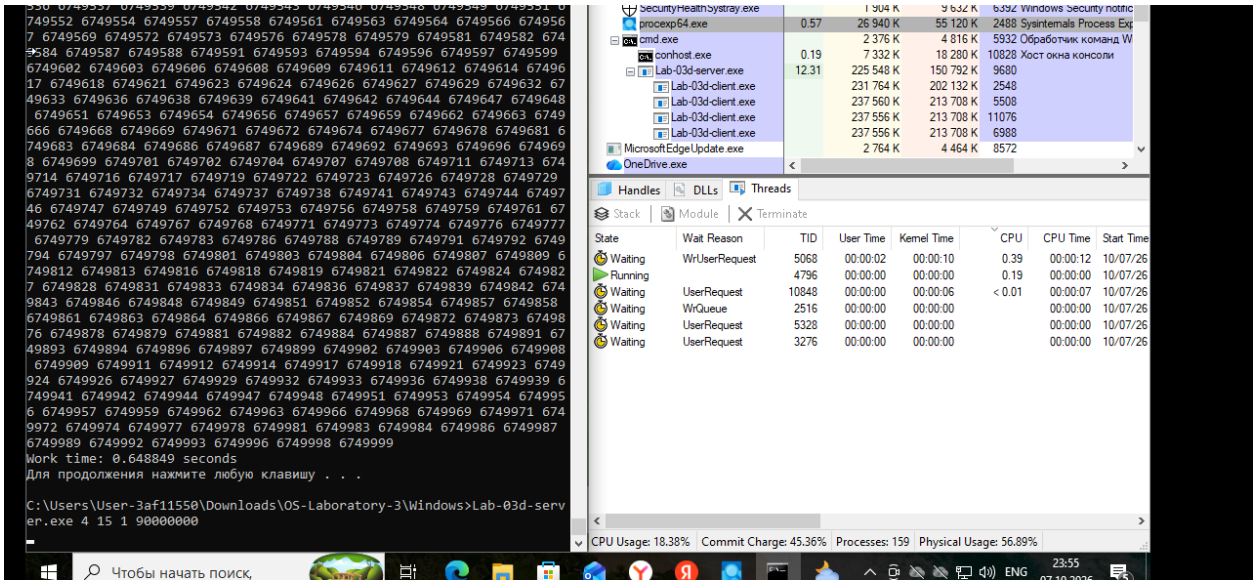



Рисунок 3.2 – Дерево процессов параллельного поиска

Дерево процессов демонстрирует одновременную работу четырех дочерних экземпляров приложения Lab-03d-client, подчиненных родительскому процессу Lab-03d-server. Наличие различных идентификаторов (PID) подтверждает, что вычислительные поддиапазоны распределены между независимыми изолированными процессами. Архитектура приложения подразумевает, что родительский процесс переходит в режим ожидания только после успешного запуска всех клиентских процессов.

На рисунке 3.3 представлена нижняя панель Handles в Process Explorer для проверки канала у сервера.

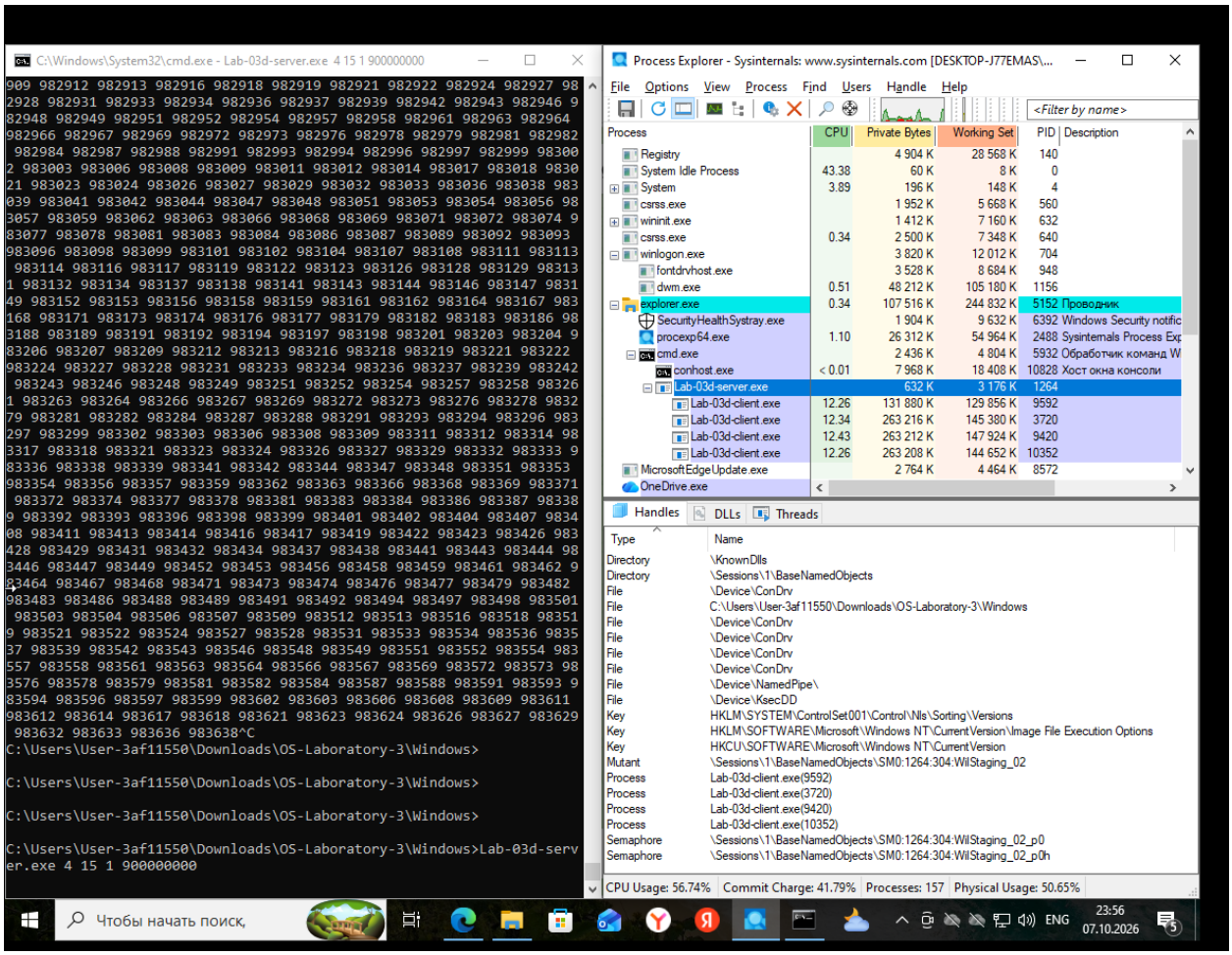



Рисунок 3.3 – Дескриптор анонимного канала в Process Explorer

В списке присутствует дескриптор типа File с именем в пространстве \Device\NamedPipe. Это представление анонимного канала в Process Explorer; рядом видны дескрипторы дочерних процессов.

## 4 Передача данных через STARTUPINFO в Windows

Для реализации межпроцессной передачи текстовой строки, исключающей использование стандартных механизмов (аргументы командной строки, переменные окружения, перенаправление потоков ввода-вывода), родительское приложение задействует структуру STARTUPINFOA. Заданная строка «Hello, my dear child!» записывается в поле lpTitle указанной структуры. После этого функция CreateProcessA инициализирует дочерний процесс в отдельном окне консоли. Дочерняя программа, в свою очередь, извлекает параметры запуска посредством вызова GetStartupInfoA и выводит полученную строку в стандартный поток вывода.

Обязательным условием данного метода является флаг создания новой консоли при вызове CreateProcessA. В этом случае ОС использует содержимое поля lpTitle в качестве заголовка нового окна, одновременно предоставляя дочернему процессу доступ к этой информации через структуру STARTUPINFOA. Иные поля структуры в рамках данного опыта не используются.

Результат выполнения программы продемонстрирован на рисунке 4.1.

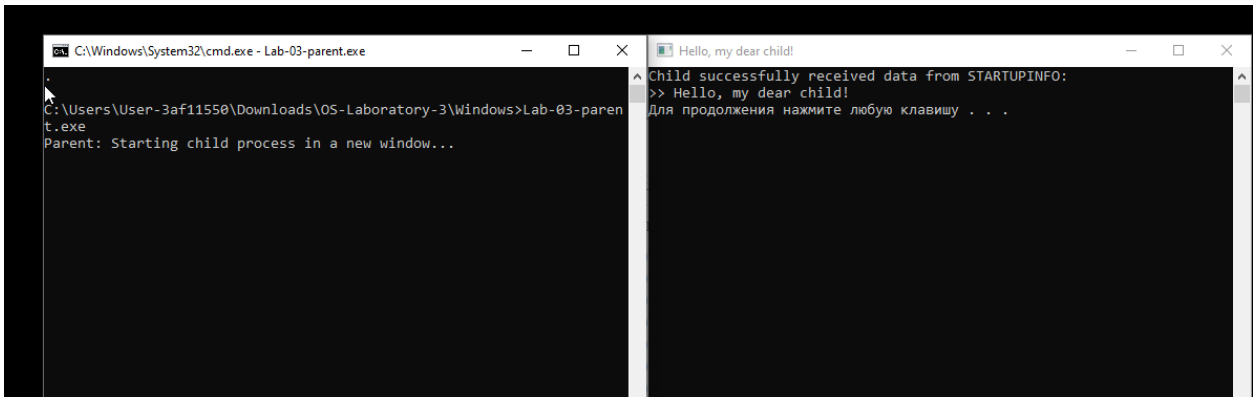



Рисунок 4.1 – Передача фразы через STARTUPINFO

В заголовке инициализированного дочернего окна и непосредственно в его консольном выводе отображается переданная строка. Отсутствие данной фразы в аргументах запуска подтверждает успешную передачу данных исключительно через поле lpTitle структуры STARTUPINFOA.

# 5 Создание и исследование процессов Linux

## 5.1 Приложения Lab-03x и Lab-03a

Реализация приложения Lab-03x для ОС Linux определяет количество итераций рабочего цикла на основе предоставляемых входных данных. Приоритетным источником значения выступает переданный аргумент командной строки (массив argv), а в случае его отсутствия считывается переменная окружения ITER_NUM. На каждой итерации программа выводит в стандартный поток вывода свой уникальный системный идентификатор (PID), после чего приостанавливает выполнение на 500 миллисекунд с помощью вызова функции usleep.

Перечень системных команд и диагностических утилит, задействованных для независимого мониторинга состояния процессов, сведен в таблицу 5.1. Прямое сопоставление полученных идентификаторов (PID) из консольного вывода программы с динамическими структурами виртуальной файловой системы /proc позволяет достоверно верифицировать работу алгоритма. Результаты анализа подтверждают корректность выделения ресурсов для создания дочерних процессов и их параллельное выполнение в операционной системе.

Таблица 5.1 – Наблюдение за процессами Linux

| Элемент | Назначение и ожидаемый результат |
|---|---|
| ps -x -o pid,ppid,stat,cmd | Показывает PID, PPID, состояние и команду. |
| cat /proc/<pid>/status | Даёт сведения Name, State, Pid, PPid и другие поля процесса. |
| ls -l /proc/<pid>/fd | Показывает ссылки pipe:[номер] для открытых файловых дескрипторов. |
| waitpid | Ожидает завершения ребёнка. |

Родительское приложение Lab-03a инициализирует два дочерних процесса системным вызовом fork и загружает в их адресное пространство исполняемый модуль Lab-03x посредством вызова execl. Первому дочернему процессу напрямую передается аргумент 10. Перед выполнением второго вызова fork родитель устанавливает значение переменной окружения ITER_NUM=15. Создание обоих дочерних экземпляров завершается до перехода родительского потока в режим ожидания через waitpid.

Вывод родителя и двух детей Lab-03a представлен на рисунке 5.1.

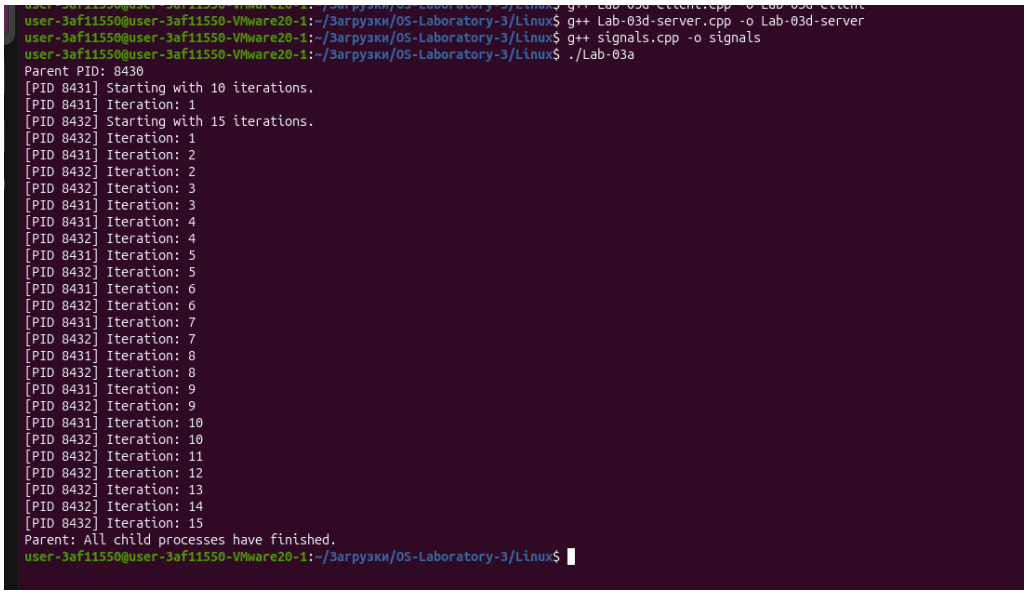



Рисунок 5.1 – Параллельный запуск двух процессов Lab-03x

Родитель сообщил свой PID, а дети вывели разные идентификаторы и выполнили соответственно 10 и 15 итераций. Завершающая строка родителя появляется после ожидания обоих процессов.

Во время отдельного запуска сведения о PID и PPID проверены командой ps и в файле /proc/<pid>/status; результат приведён на рисунке 5.2.

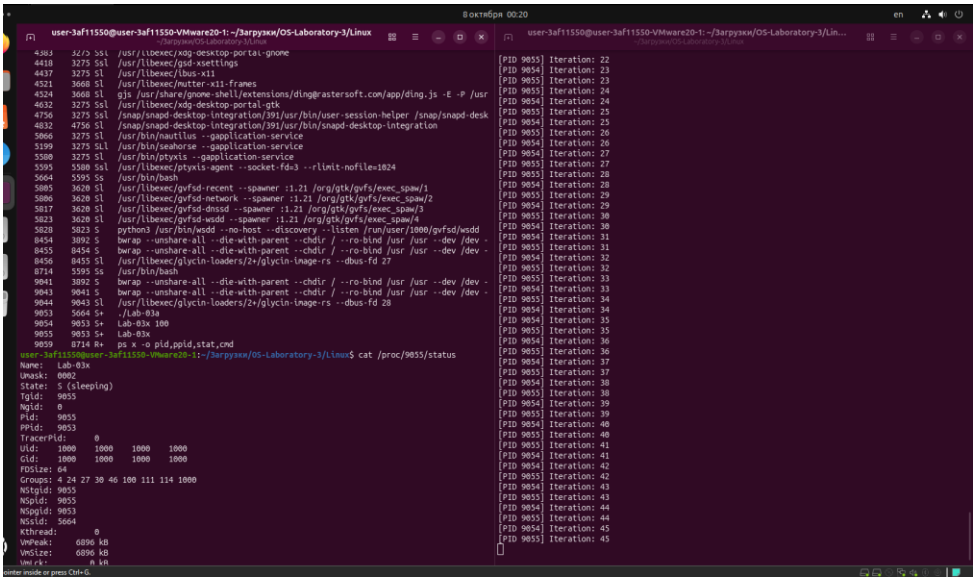



Рисунок 5.2 – Сведения о процессах через ps и каталог proc

Для показанного ребёнка PID=9055, PPID=9053. Эти значения совпадают в ps и /proc; данный снимок сделан при другом запуске Lab-03a, поэтому PID отличаются от рисунка 5.1.

## 6 Анонимные каналы Linux

Linux-сервер создаёт один pipe до цикла fork. Каждый ребёнок закрывает конец чтения и через dup2 направляет стандартный вывод в конец записи, после чего execl загружает Lab-03d-client. Родитель закрывает свой конец записи, читает общий поток до EOF и ожидает всех детей через waitpid. В программе не предусмотрена синхронизация записи клиентов, как и указано в задании.

Клиент записывает числа через write(STDOUT_FILENO, ...), не зная числового значения исходного дескриптора канала. Общий поток затем разбирается сервером на значения, разделённые пробелами.

Для проверки результата Linux-сервера использованы M=12, K=4 и диапазон [1, 31]. Вывод параметров, найденных чисел и времени работы приведён на рисунке 6.1.

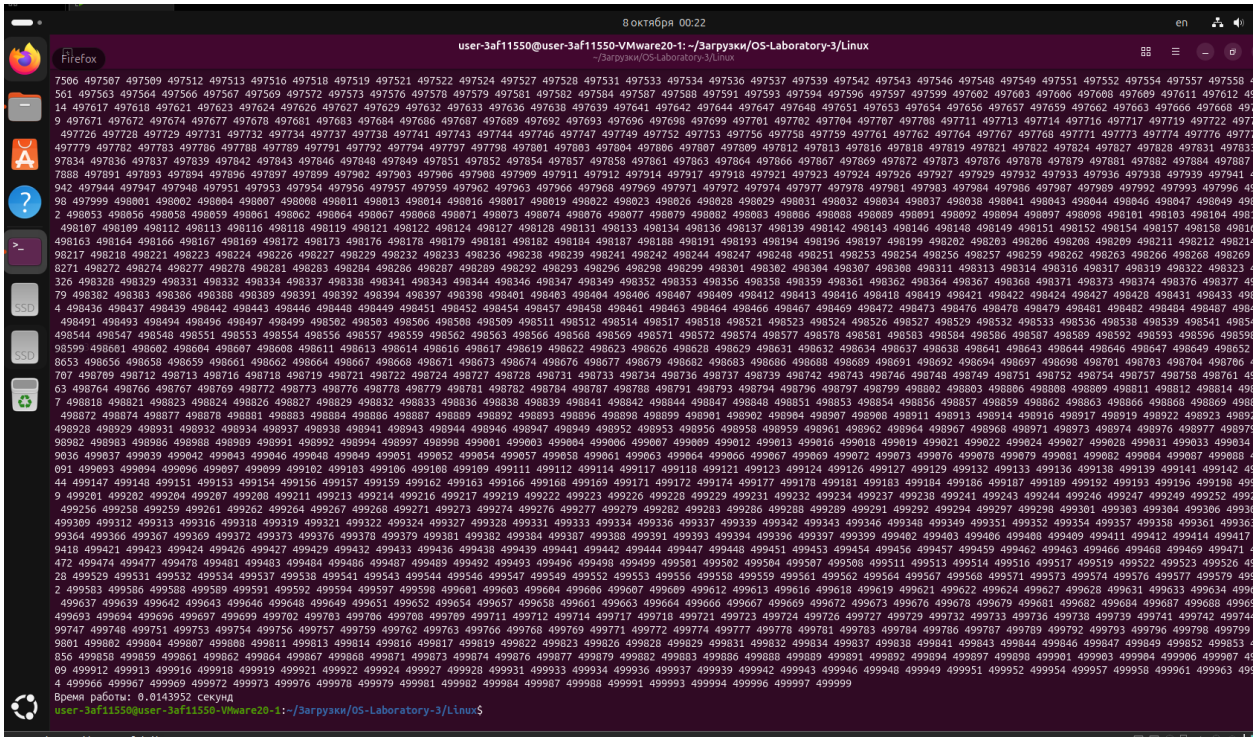



Рисунок 6.1 – Результат параллельного поиска Linux

В этом диапазоне результатом являются те же 11 чисел, что и в Windows-версии. Совпадение двух списков позволяет проверить логику поиска независимо от способа создания процессов.

Связь стандартного вывода клиента с каналом проверена через каталог /proc. Сведения о дескрипторах родителя и ребёнка приведены на рисунке 6.2.

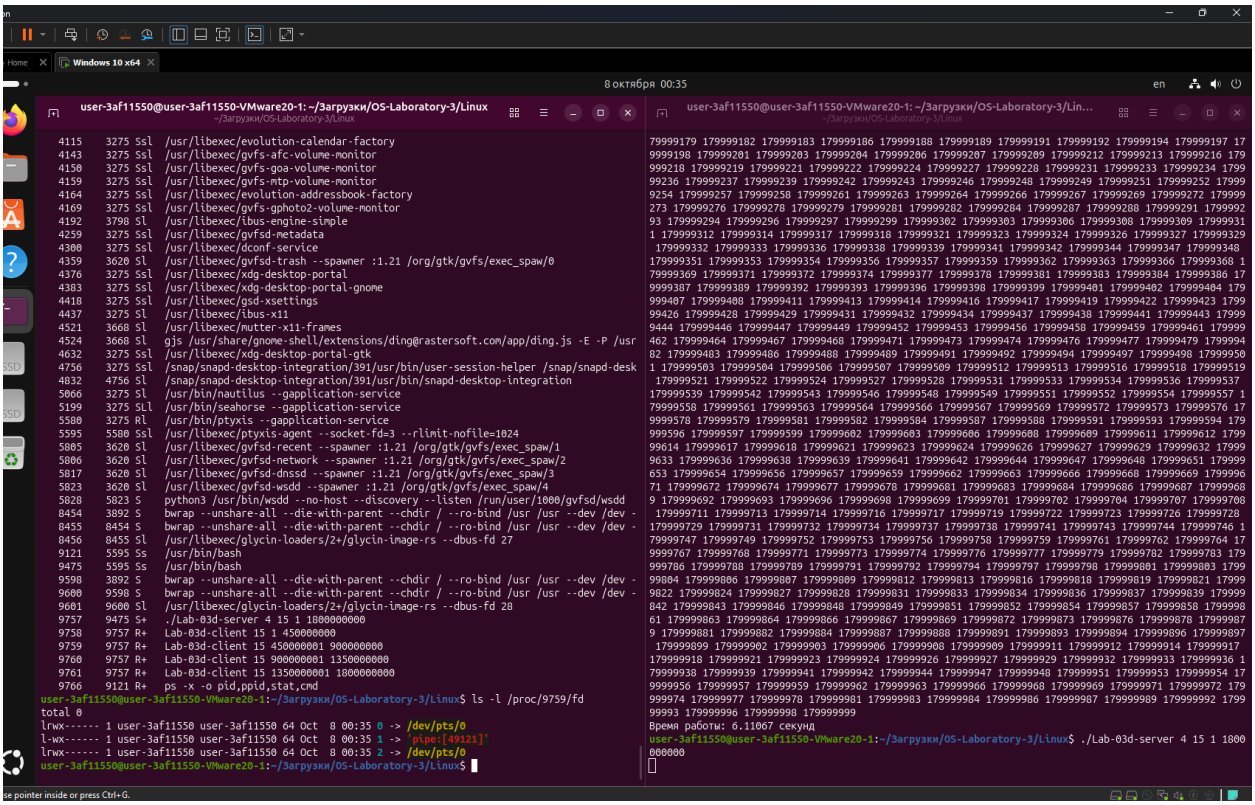



Рисунок 6.2 – Дескрипторы канала в proc

Ссылки вида pipe:[номер] показывают, что файловые дескрипторы процессов относятся к каналу. У клиента такой ссылкой должен быть стандартный вывод с номером 1; у родителя — читающий конец того же канала.

## 7 Обработка сигналов Linux

Программа signals устанавливает обработчики SIGUSR1 и SIGUSR2 функцией signal, игнорирует SIGINT и оставляет для SIGTERM действие по умолчанию. После вывода PID она ожидает сигналы в pause. Поэтому пользовательские сигналы вызывают сообщения, Ctrl+C не завершает процесс, а SIGTERM прекращает его работу.

Проверяемые сигналы и поведение программы приведены в таблице 7.1.

Таблица 7.1 – Сигналы тестового приложения

| Элемент | Назначение и ожидаемый результат |
|---|---|
| SIGUSR1 | Пользовательский сигнал; обработчик выводит Received USR1 signal. |
| SIGUSR2 | Второй пользовательский сигнал; обработчик выводит Received USR2 signal. |
| SIGINT | Игнорируется, поэтому Ctrl+C не останавливает приложение. |
| SIGTERM | Обрабатывается действием по умолчанию и завершает процесс. |

Сначала сигналы отправлены работающему в фоне процессу командой kill; затем игнорирование Ctrl+C отдельно проверено при запуске на переднем плане.

Реакция фонового процесса на SIGUSR1, SIGUSR2, SIGINT и SIGTERM показана на рисунке 7.1.

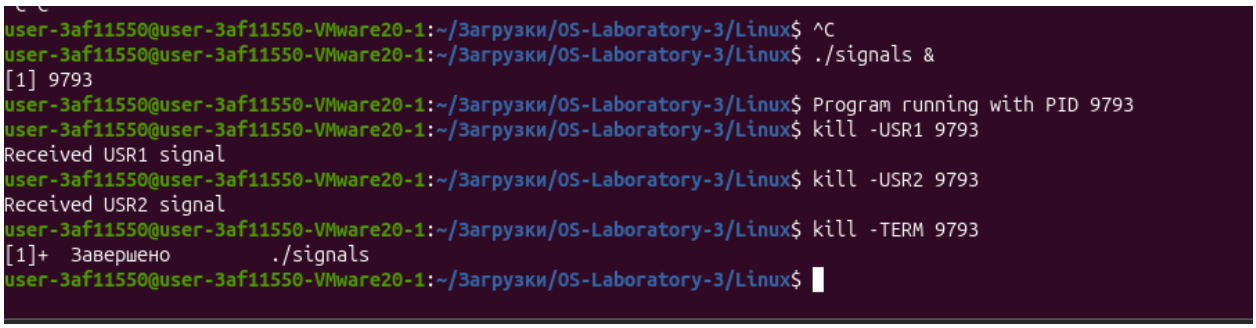



Рисунок 7.1 – Обработка USR1, USR2, INT и TERM командами kill

После SIGUSR1 и SIGUSR2 появляются сообщения обработчиков. SIGINT оставляет процесс работающим, а SIGTERM завершает его, что видно по возвращению приглашения оболочки и сообщению о завершённом фоновом задании.

Во втором опыте signals запущена на переднем плане. Нажатие Ctrl+C не возвращает приглашение оболочки; процесс завершается только после отправки SIGTERM из соседнего терминала (рисунок 7.2).

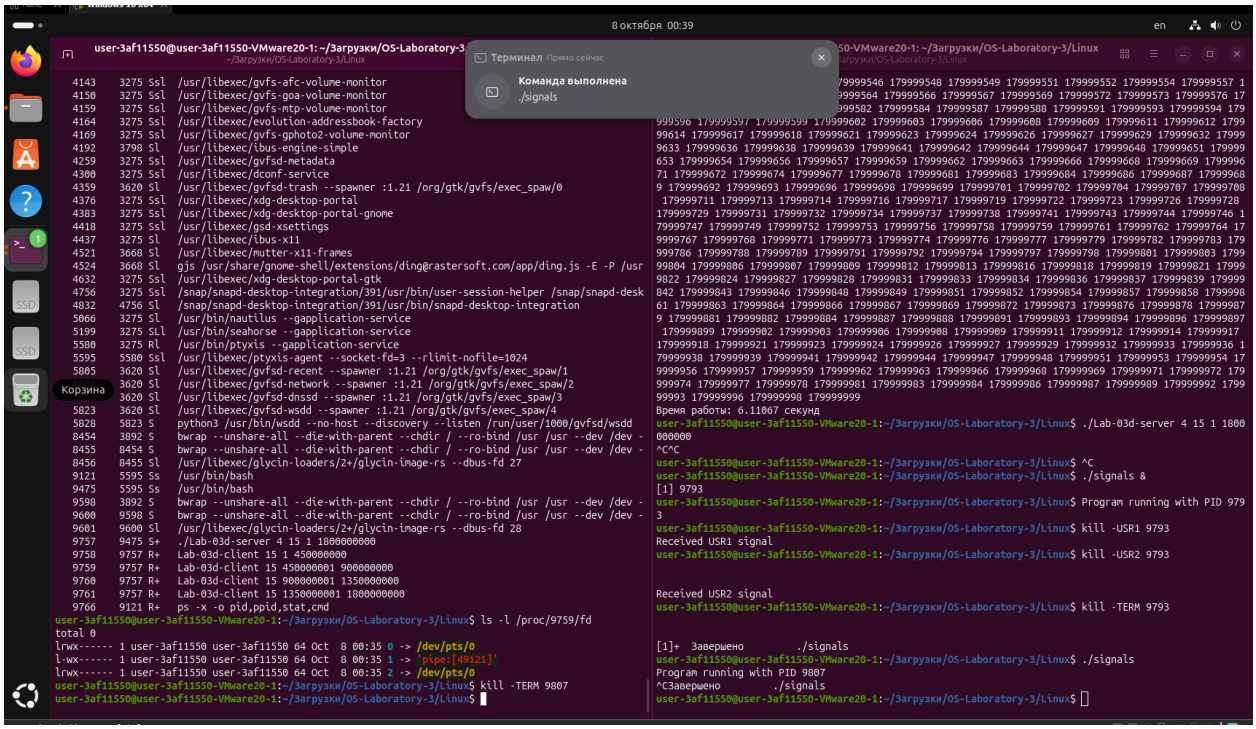



Рисунок 7.2 – Проверка Ctrl+C при запуске signals на переднем плане

Так проверяется именно сочетание клавиш Ctrl+C, а не только отправка того же сигнала командой kill. PID в обоих терминалах должен относиться к одному запуску программы.

## Заключение

В работе созданы и исследованы процессы Windows и Linux. Проверены три формы вызова CreateProcessA, передача значения через аргументы и окружение, связка fork и execl, просмотр PID и PPID системными средствами. Результаты этих опытов сопоставлены с выводом программ.

Поиск взаимно простых чисел показал, как несколько дочерних процессов передают результаты родителю через анонимный канал. Опыт с STARTUPINFOA позволил передать строку без аргументов и переменных окружения. В Linux проверено назначение обработчиков SIGUSR1 и SIGUSR2, игнорирование SIGINT и завершение по SIGTERM. Таким образом, результаты наблюдения за процессами согласуются с выбранными способами их создания и взаимодействия.

**Список использованных источников**

1 Microsoft Learn. CreateProcessA function:
https://learn.microsoft.com/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa
2 Microsoft Learn. Process and Thread Functions:
https://learn.microsoft.com/windows/win32/procthread/process-and-thread-functions
3 Microsoft Learn. Creating a Child Process with Redirected Input and Output:
https://learn.microsoft.com/windows/win32/procthread/creating-a-child-process-with-redirected-input-and-output
4 Microsoft Learn. Tool Help Library:
https://learn.microsoft.com/windows/win32/toolhelp/tool-help-library
5 Microsoft Sysinternals. Process Explorer:
https://learn.microsoft.com/sysinternals/downloads/process-explorer
6 The Open Group Base Specifications. fork, exec and wait:
https://pubs.opengroup.org/onlinepubs/9699919799/
7 Linux manual pages. proc, pipe, sigaction and signal:
https://man7.org/linux/man-pages/

**Приложение А Исходные тексты программ Windows**

Листинг А.1 – Lab-03x.cpp

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <string>
#include <cstdlib>
#include <windows.h>

using namespace std;

int main(int argc, char* argv[])
{
   int iteration_count;

   if (argc > 1)
   {
      iteration_count = stoi(argv[1]);
      cout << "The number of iterations was successfully obtained from the
command-line argument.\nNumber of iteration: " << iteration_count << "\n";
   }
   else
   {
      const char* env_parameter = getenv("ITER_NUM");
      if (env_parameter != nullptr)
      {
         iteration_count = stoi(env_parameter);
         cout << "The number of iterations was successfully obtained from the
environment variable.\nNumber of iteration: " << iteration_count << "\n";
      }
      else
      {
         cerr << "Failed to obtain the number of loop iterations from either the
command-line argument or the environment variable.\nSet an environment
variable named ITER_NUM or pass the number of iterations using the
command-line argument -i n\n";
         ExitProcess(1);
      }
   }

   DWORD pid = GetCurrentProcessId();

   for (int i = 0; i < iteration_count; i++)
   {
```

```
        cout << "PID: " << pid << " | Iteration: " << i + 1 << "\n";
        Sleep(500);
    }
    return 0;
}
```

Листинг А.2 – Lab-03a.cpp

```
#include <iostream>
#include <windows.h>
#include <vector>
#include <string>

using namespace std;

int main()
{
    string first_call_string = "Lab-03x.exe 50";
    char second_call_string[] = "Lab-03x.exe 50";
    string third_call_string = "Lab-03x.exe";
    char third_argument[] = " 50";

    STARTUPINFOA startup_info;
    PROCESS_INFORMATION process_info;
    vector<HANDLE> hProcesses;

    cout << "First process starting...\n";

    ZeroMemory(&startup_info, sizeof(startup_info));
    startup_info.cb = sizeof(startup_info);

    /*
        1. char* на команду для командной строки
        2. Аргументы командой строки
        3. Права доступа дочернего процесса
        4. Настройки безопастности  первичного потока процесса
        5. Наследовать ли открытые файлы, каналы.. от родителя
        6. Флаги создания
        7. Блок переменных окружения
        8. Рабочая папка
        9. Блок для информации о запуске
        10. Блок на стркутуру, куда запишем pid
    */
    if (CreateProcessA(first_call_string.c_str(), NULL, NULL, NULL, FALSE, 0,
NULL, NULL, &startup_info, &process_info))
```

```
    {
        cout << "Successfull! PID: " << process_info.dwProcessId << "\n";
        hProcesses.push_back(process_info.hProcess);
        CloseHandle(process_info.hThread);
    }
    else
    {
        cout << "Error! Error code: " << GetLastError() << " (Failed to create
process).\n";
    }


    cout << "Second process starting...\n";
    ZeroMemory(&startup_info, sizeof(startup_info));
    startup_info.cb = sizeof(startup_info);

    if (CreateProcessA(NULL, second_call_string, NULL, NULL, FALSE, 0,
NULL, NULL, &startup_info, &process_info))
    {
        cout << "Successfull! PID: " << process_info.dwProcessId << "\n";
        hProcesses.push_back(process_info.hProcess);
        CloseHandle(process_info.hThread);
    }
    else
    {
        cout << "Error! Error code: " << GetLastError() << " (Failed to create
process).\n";
    }

    cout << "Third process starting...\n";
    ZeroMemory(&startup_info, sizeof(startup_info));
    startup_info.cb = sizeof(startup_info);

    if (CreateProcessA(third_call_string.c_str(), third_argument, NULL, NULL,
FALSE, 0, NULL, NULL, &startup_info, &process_info))
    {
        cout << "Successfull! PID: " << process_info.dwProcessId << "\n";
        hProcesses.push_back(process_info.hProcess);
        CloseHandle(process_info.hThread);
    }
    else
    {
        cout << "Error! Error code: " << GetLastError() << " (Failed to create
process).\n";
```

```
    }

    if (!hProcesses.empty())
    {
        cout << "\nWait to complete child process...\n";
        WaitForMultipleObjects(hProcesses.size(), hProcesses.data(), TRUE,
INFINITE);
        cout << "All process are completed.\n";

        for (HANDLE h : hProcesses)
        {
            CloseHandle(h);
        }
    }

    system("pause");
    return 0;
}
```

Листинг А.3 – Lab-03b.cpp

```
#include <iostream>
#include <windows.h>
#include <vector>
#include <string>

using namespace std;

int main()
{
    string first_call_string = "Lab-03x.exe";
    char second_call_string[] = "Lab-03x.exe 50";
    string third_call_string = "Lab-03x.exe";
    char third_argument[] = " 50";

    STARTUPINFOA startup_info;
    PROCESS_INFORMATION process_info;
    vector<HANDLE> hProcesses;

    cout << "First process starting...\n";

    ZeroMemory(&startup_info, sizeof(startup_info));
    startup_info.cb = sizeof(startup_info);

    /*
```

```
      1. char* на команду для командной строки
      2. Аргументы командой строки
      3. Права доступа дочернего процесса
      4. Настройки безопастности  первичного потока процесса
      5. Наследовать ли открытые файлы, каналы.. от родителя
      6. Флаги создания
      7. Блок переменных окружения
      8. Рабочая папка
      9. Блок для информации о запуске
      10. Блок на стркутуру, куда запишем pid
   */
   if (CreateProcessA(first_call_string.c_str(), NULL, NULL, NULL, FALSE, 0,
NULL, NULL, &startup_info, &process_info))
   {
      cout << "Successfull! PID: " << process_info.dwProcessId << "\n";
      hProcesses.push_back(process_info.hProcess);
      CloseHandle(process_info.hThread);
   }
   else
   {
      cout << "Error! Error code: " << GetLastError() << " (Failed to create
process).\n";
   }


   cout << "Second process starting...\n";
   ZeroMemory(&startup_info, sizeof(startup_info));
   startup_info.cb = sizeof(startup_info);

   if (CreateProcessA(NULL, second_call_string, NULL, NULL, FALSE, 0,
NULL, NULL, &startup_info, &process_info))
   {
      cout << "Successfull! PID: " << process_info.dwProcessId << "\n";
      hProcesses.push_back(process_info.hProcess);
      CloseHandle(process_info.hThread);
   }
   else
   {
      cout << "Error! Error code: " << GetLastError() << " (Failed to create
process).\n";
   }

   cout << "Third process starting...\n";
   ZeroMemory(&startup_info, sizeof(startup_info));
```

```
    startup_info.cb = sizeof(startup_info);

    if (SetEnvironmentVariableA("ITER_NUM", "60"))
    {
        cout << "Local environment variable ITER_NUM is set to 60.\n";
    }

    if (CreateProcessA(third_call_string.c_str(), NULL, NULL, NULL, FALSE, 0,
NULL, NULL, &startup_info, &process_info))
    {
        cout << "Successfull! PID: " << process_info.dwProcessId << "\n";
        hProcesses.push_back(process_info.hProcess);
        CloseHandle(process_info.hThread);
    }
    else
    {
        cout << "Error! Error code: " << GetLastError() << " (Failed to create
process).\n";
    }

    if (!hProcesses.empty())
    {
        cout << "\nWait to complete child process...\n";
        WaitForMultipleObjects(hProcesses.size(), hProcesses.data(), TRUE,
INFINITE);
        cout << "All process are completed.\n";

        for (HANDLE h : hProcesses)
        {
            CloseHandle(h);
        }
    }

    system("pause");
    return 0;
}
```

Листинг А.4 – Lab-03c.cpp

```
#include <iostream>
#include <windows.h>
#include <tlhelp32.h>
#include <iomanip>

using namespace std;
```

```
int main()
{
   HANDLE hProcessSnap =
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);

   if (hProcessSnap == INVALID_HANDLE_VALUE)
   {
      cout << "Error: Failed to create snapshot. Error code: " << GetLastError()
<< "\n";
      return 1;
   }

   PROCESSENTRY32 process_info;
   process_info.dwSize = sizeof(PROCESSENTRY32);

   if (!Process32First(hProcessSnap, &process_info))
   {
      cout << "Error: Process32First failed. Error code: " << GetLastError() <<
"\n";
      CloseHandle(hProcessSnap);
      return 1;
   }

   cout << left << setw(10) << "PID" << setw(15) << "Parent PID" << "Process
Name" << "\n";
   cout << string(50, '-') << "\n";

   do
   {
      wcout << left << setw(10) << process_info.th32ProcessID << setw(15) <<
process_info.th32ParentProcessID << process_info.szExeFile << L"\n";

   } while (Process32Next(hProcessSnap, &process_info));

   CloseHandle(hProcessSnap);
   cout << "\n";

   system("pause");
   return 0;
}
```

Листинг А.5 – Lab-03d-client.cpp

```
#include <iostream>
```

```
#include <windows.h>
#include <string>
#include <sstream>

using namespace std;

// Функция нахождения нод
long long GetGreatestCommonDivisor(long long a, long long b)
{
   while (b)
   {
      a %= b;
      swap(a, b);
   }
   return a;
}

int main(int argc, char* argv[])
{
   if (argc < 4)
   {
      return 1;
   }

   long long M = stoll(argv[1]);
   long long L = stoll(argv[2]);
   long long R = stoll(argv[3]);

   stringstream ss;
   for (long long n = L; n <= R; n++)
   {
      if (GetGreatestCommonDivisor(n, M) == 1)
      {
         ss << n << " ";
      }
   }

   string result = ss.str();

   HANDLE h_out = GetStdHandle(STD_OUTPUT_HANDLE);
   if (h_out == INVALID_HANDLE_VALUE)
   {
      ExitProcess(GetLastError());
   }
```

```
    DWORD written;
    if (!result.empty())
    {
        WriteFile(h_out, result.c_str(), result.length(), &written, NULL);
    }

    return 0;
}
```

Листинг А.6 – Lab-03d-server.cpp

```
#include <iostream>
#include <windows.h>
#include <string>
#include <vector>
#include <sstream>
#include <chrono>

using namespace std;

int main(int argc, char* argv[]) {
    if (argc < 5)
    {
        cout << "Usage: Lab-03d-server <K> <M> <L> <R>\n";
        return 1;
    }

    int K = stoi(argv[1]);
    long long M = stoll(argv[2]);
    long long L = stoll(argv[3]);
    long long R = stoll(argv[4]);

    if (K <= 0 || L > R || M <= 0)
    {
        cout << "Invalid arguments.\n";
        return 1;
    }

    HANDLE hRead, hWrite;
    SECURITY_ATTRIBUTES sa;         // Настройки безопастности
    ZeroMemory(&sa, sizeof(sa));
    sa.nLength = sizeof(sa);
    sa.bInheritHandle = TRUE;       // Разрешаем дочерним процессам
наследовать дескрипторы
```

```
    sa.lpSecurityDescriptor = NULL; // Стандартные пд

    if (!CreatePipe(&hRead, &hWrite, &sa, 0))
    {
        cout << "Error: CreatePipe failed. Code: " << GetLastError() << "\n";
        return 1;
    }

    SetHandleInformation(hRead, HANDLE_FLAG_INHERIT, 0); // - Дескр.
чтения

    long long range = R - L + 1;
    long long step = range / K;

    vector<HANDLE> hProcesses;
    long long current_L = L;

    auto start_time = chrono::high_resolution_clock::now();

    for (int i = 0; i < K; i++)
    {
        long long current_R = current_L + step - 1;

        if (current_L > current_R)
        {
            break;
        }

        string cmdStr = "Lab-03d-client.exe " + to_string(M) + " " +
to_string(current_L) + " " + to_string(current_R);
        char cmd[256];
        strcpy_s(cmd, cmdStr.c_str());

        STARTUPINFOA si;
        PROCESS_INFORMATION pi;
        ZeroMemory(&si, sizeof(si));
        si.cb = sizeof(si);

        si.hStdOutput = hWrite;
        si.dwFlags |= STARTF_USESTDHANDLES;

        if (CreateProcessA(NULL, cmd, NULL, NULL, TRUE, 0, NULL, NULL,
&si, &pi))
        {
```

```
            hProcesses.push_back(pi.hProcess);
            CloseHandle(pi.hThread);
        }
        else
        {
            cout << "Failed to create process " << i << ". Error: " << GetLastError()
<< "\n";
        }

        current_L = current_R + 1;
    }

    CloseHandle(hWrite);

    string all_results = "";
    char buffer[4096];
    DWORD bytesRead;

    while (ReadFile(hRead, buffer, sizeof(buffer) - 1, &bytesRead, NULL) &&
bytesRead > 0)
    {
        buffer[bytesRead] = '\0';
        all_results += buffer;
    }

    if (!hProcesses.empty())
    {
        WaitForMultipleObjects(hProcesses.size(), hProcesses.data(), TRUE,
INFINITE);
        for (HANDLE h : hProcesses)
        {
            CloseHandle(h);
        }
    }
    CloseHandle(hRead);

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double> diff = end_time - start_time;

    int total_count = 0;
    stringstream result_stream(all_results);
    string temp;
    while (result_stream >> temp)
    {
```

```
        total_count++;
    }

    cout << "--- Result ---\n";
    cout << "Number M: " << M << "\n";
    cout << "Range: [" << L << ", " << R << "]\n";
    cout << "Process count K: " << K << "\n";
    cout << "General count of founded numbers: " << total_count << "\n";
    cout << "Found numbers: " << all_results << "\n";
    cout << "Work time: " << diff.count() << " seconds\n";

    system("pause");
    return 0;
}
```

Листинг А.7 – Lab-03-parent.cpp

```
#include <iostream>
#include <windows.h>

using namespace std;

int main()
{
    STARTUPINFOA startup_info;
    PROCESS_INFORMATION process_info;

    ZeroMemory(&startup_info, sizeof(startup_info));
    startup_info.cb = sizeof(startup_info);

    char secret_message[] = "Hello, my dear child!";
    startup_info.lpTitle = secret_message;

    char child_app[] = "Lab-03-child.exe";

    cout << "Parent: Starting child process in a new window...\n";

    if (CreateProcessA(NULL, child_app, NULL, NULL, FALSE,
CREATE_NEW_CONSOLE, NULL, NULL, &startup_info, &process_info))
    {
        WaitForSingleObject(process_info.hProcess, INFINITE);
        CloseHandle(process_info.hThread);
        CloseHandle(process_info.hProcess);
    }
    else
```

```
    {
        cout << "Error creating process. Code: " << GetLastError() << "\n";
    }

    return 0;
}
```

Листинг А.8 – Lab-03-child.cpp

```
#include <iostream>
#include <windows.h>

using namespace std;

int main()
{
    STARTUPINFOA si;
    ZeroMemory(&si, sizeof(si));

    GetStartupInfoA(&si);

    if (si.lpTitle != NULL)
    {
        cout << "Child successfully received data from STARTUPINFO:\n";
        cout << ">> " << si.lpTitle << "\n";
    }
    else
    {
        cout << "Child: No message found in lpTitle.\n";
    }

    system("pause");
    return 0;
}
```

**Приложение Б Исходные тексты программ Linux**

Листинг Б.1 – Lab-03x.cpp

```
#include <iostream>
#include <cstdlib>
#include <unistd.h>

using namespace std;

int main(int argc, char* argv[])
{
   int iterations = 0;

   if (argc > 1)
   {
      iterations = atoi(argv[1]);
   }
   else
   {
      char* env_iter = getenv("ITER_NUM");
      if (env_iter != NULL)
      {
         iterations = atoi(env_iter);
      }
      else
      {
         cerr << "Error: No ITER_NUM or command line argument provided.\n";
         exit(139);
      }
   }

   cout << "[PID " << getpid() << "] Starting with " << iterations << "
iterations.\n";

   for (int i = 0; i < iterations; ++i)
   {
      cout << "[PID " << getpid() << "] Iteration: " << i + 1 << "\n";
      usleep(500000);
   }

   return 0;
}
```

Листинг Б.2 – Lab-03a.cpp

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>
#include <cstdlib>

using namespace std;

int main()
{
   pid_t pid1, pid2;

   cout << "Parent PID: " << getpid() << "\n";

   pid1 = fork();
   if (pid1 == 0)
   {
      execl("./Lab-03x", "Lab-03x", "10", NULL);

      cerr << "Failed to start first child.\n";
      exit(1);
   }

   setenv("ITER_NUM", "15", 1);

   pid2 = fork();
   if (pid2 == 0)
   {
      execl("./Lab-03x", "Lab-03x", NULL);

      cerr << "Failed to start second child.\n";
      exit(1);
   }

   int status;
   if (pid1 > 0) waitpid(pid1, &status, 0);
   if (pid2 > 0) waitpid(pid2, &status, 0);

   cout << "Parent: All child processes have finished.\n";
   return 0;
}
```

Листинг Б.3 – Lab-03d-client.cpp

```
#include <iostream>
```

```
#include <string>
#include <sstream>
#include <unistd.h>

using namespace std;

long long gcd(long long a, long long b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}

int main(int argc, char* argv[]) {
    if (argc < 4) return 1;

    long long M = stoll(argv[1]);
    long long L = stoll(argv[2]);
    long long R = stoll(argv[3]);

    stringstream ss;
    for (long long n = L; n <= R; ++n) {
        if (gcd(n, M) == 1) {
            ss << n << " ";
        }
    }

    string result = ss.str();
    if (!result.empty()) {
        write(STDOUT_FILENO, result.c_str(), result.length());
    }

    return 0;
}
```

Листинг Б.4 – Lab-03d-server.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <unistd.h>
#include <sys/wait.h>
#include <chrono>
```

```
using namespace std;

int main(int argc, char* argv[]) {
    if (argc < 5) {
        cout << "Usage: ./Lab-03d-server <K> <M> <L> <R>\n";
        return 1;
    }

    int K = stoi(argv[1]);
    long long M = stoll(argv[2]);
    long long L = stoll(argv[3]);
    long long R = stoll(argv[4]);

    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        return 1;
    }

    long long range = R - L + 1;
    long long step = range / K;
    long long rem = range % K;

    vector<pid_t> pids;
    long long current_L = L;
    auto start_time = chrono::high_resolution_clock::now();

    for (int i = 0; i < K; ++i) {
        long long current_R = current_L + step - 1;

        if (K == 1) current_R += rem;
        else if (i == 1) current_R += rem;

        if (current_L > current_R) break;

        pid_t pid = fork();
        if (pid == 0) {
            close(pipefd[0]);

            dup2(pipefd[1], STDOUT_FILENO);
            close(pipefd[1]);

            string str_M = to_string(M);
```

```
            string str_L = to_string(current_L);
            string str_R = to_string(current_R);

            execl("./Lab-03d-client", "Lab-03d-client", str_M.c_str(), str_L.c_str(),
str_R.c_str(), NULL);

            perror("execl failed");
            exit(1);
        }
        else if (pid > 0) {
            pids.push_back(pid);
        }
        current_L = current_R + 1;
    }

    close(pipefd[1]);

    string all_results = "";
    char buffer[4096];
    ssize_t bytesRead;

    while ((bytesRead = read(pipefd[0], buffer, sizeof(buffer) - 1)) > 0) {
        buffer[bytesRead] = '\0';
        all_results += buffer;
    }
    close(pipefd[0]);

    for (pid_t pid : pids) {
        waitpid(pid, NULL, 0);
    }

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double> diff = end_time - start_time;

    int total_count = 0;
    stringstream ss(all_results);
    string temp;
    while (ss >> temp) total_count++;

    cout << "--- Результы работы сервера ---\n";
    cout << "Число M: " << M << "\n";
    cout << "Диапазон: [" << L << ", " << R << "]\n";
    cout << "Количество процессов K: " << K << "\n";
    cout << "Общее количество найденных чисел: " << total_count << "\n";
```

```
    cout << "Найденные числа: " << all_results << "\n";
    cout << "Время работы: " << diff.count() << " секунд\n";

    return 0;
}
```

Листинг Б.5 – signals.cpp

```
#include <iostream>
#include <unistd.h>
#include <signal.h>

using namespace std;

void signal_handler(int signum) {
    if (signum == SIGUSR1) {
        cout << "Received USR1 signal\n";
    }
    else if (signum == SIGUSR2) {
        cout << "Received USR2 signal\n";
    }
}

int main() {
    signal(SIGUSR1, signal_handler);
    signal(SIGUSR2, signal_handler);

    signal(SIGINT, SIG_IGN);

    cout << "Program running with PID " << getpid() << "\n";

    while (true) {
        pause();
    }

    return 0;
}
```